

THE COMPLETE COMPUTER SCIENCE ROADMAP

Computers ki Bunyaad se Artificial Intelligence aur Quantum Computing tak

MIT · Stanford · CMU · Oxford · Cambridge Level — Mukammal Ordered Topics

Yeh PDF Computer Science ka **mukammal ordered roadmap** hai — bilkul shuruaat ('computer kya hota hai') se lekar MIT, Stanford, CMU ke PhD research frontiers tak. Isme **Programming, Data Structures, Algorithms, Systems, Networks, Databases, AI/ML, Theory of Computation, Security, aur Research-Level CS** — sab shamil hain. Har topic pichle topic ki bunyaad par bana hai. Koi cheez nahi chodi gayi. Yeh roadmap us bacche ke liye bhi hai jo sirf 'hello world' likhna chahta hai, aur us researcher ke liye bhi jo next Google ya DeepMind banana chahta hai.

LEVEL	STAGE	FOCUS
Level 1	Age 4-7	Computers, internet, logical thinking — no coding yet
Level 2	Age 7-10	Block coding, basic algorithms, digital literacy
Level 3	Age 10-13	Python basics, web basics, data, logic
Level 4	Age 13-16	Programming fluency, data structures, IGCSE CS
Level 5	Age 16-18	A-Level CS: OOP, algorithms, computer systems
Level 6	Age 18-21	Undergraduate: algorithms, systems, theory, ML intro
Level 7	Age 21-23	Advanced: compilers, distributed systems, AI/ML deep
Level 8	Age 23-25	Graduate/MSc: research-level CS
Level 9	Age 25+	PhD: frontier research — MIT/Stanford/CMU level

1.1 What is a Computer?

- 1.1.1 Computers are machines that follow instructions
- 1.1.2 Parts of a computer: screen, keyboard, mouse, CPU, memory
- 1.1.3 Tablet, phone, laptop, desktop -- all are computers
- 1.1.4 Input and output devices
- 1.1.5 What computers can and cannot do
- 1.1.6 Computers need electricity and instructions

1.2 Computational Thinking -- Foundations

- 1.2.1 Algorithms: step-by-step instructions (recipe example)
- 1.2.2 Sequencing: order matters
- 1.2.3 Patterns: spotting and using them
- 1.2.4 Decomposition: breaking big problems into small parts
- 1.2.5 Abstraction: ignoring unnecessary details
- 1.2.6 Debugging: finding and fixing mistakes

1.3 Digital Safety

- 1.3.1 Personal information: never share name, address, phone online
- 1.3.2 Trusted adults: tell a parent if something feels wrong online
- 1.3.3 Screen time: healthy limits
- 1.3.4 Kind behaviour online: cyberbullying is wrong
- 1.3.5 Passwords: keep them secret

1.4 Basic Computer Use

- 1.4.1 Mouse skills: click, double-click, drag
- 1.4.2 Keyboard: finding letters, spacebar, enter, backspace
- 1.4.3 Opening and closing applications
- 1.4.4 Saving a file
- 1.4.5 Drawing and painting on a computer
- 1.4.6 Printing a document

** Use ScratchJr (free app) for age 5-7. Block-based coding builds logical thinking visually.*

2.1 Block-Based Programming (Scratch)

- 2.1.1 Scratch interface: stage, sprites, scripts, costumes, sounds
- 2.1.2 Motion blocks: move, turn, go to position
- 2.1.3 Events: when green flag clicked, when key pressed
- 2.1.4 Control: wait, repeat, forever, if-then
- 2.1.5 Looks: say, think, change costume, show/hide
- 2.1.6 Sound: play sound, stop all sounds
- 2.1.7 Sensing: touching, mouse position, ask and answer
- 2.1.8 Variables: create, set, change, display
- 2.1.9 Simple animation project
- 2.1.10 Simple game project: catch the falling object
- 2.1.11 Simple story project: interactive narrative
- 2.1.12 Nested loops and conditions
- 2.1.13 Broadcasting messages between sprites
- 2.1.14 Cloning sprites
- 2.1.15 Lists (arrays) in Scratch

2.2 Algorithms and Logic

- 2.2.1 Flowcharts: start, end, process, decision, arrows
- 2.2.2 Drawing flowcharts for daily activities
- 2.2.3 Boolean logic: true/false, yes/no decisions
- 2.2.4 AND, OR, NOT gates -- concept with examples
- 2.2.5 Selection: if this then that
- 2.2.6 Iteration: doing something again and again
- 2.2.7 Searching: find a name in a list
- 2.2.8 Sorting: arrange numbers in order
- 2.2.9 Efficiency: fewer steps = better algorithm

2.3 Data and Information

- 2.3.1 Data: raw facts (numbers, words, images)
- 2.3.2 Information: data with meaning
- 2.3.3 How computers store data: binary concept
- 2.3.4 Bits and bytes: 1s and 0s
- 2.3.5 Files and file types: .txt, .jpg, .mp3
- 2.3.6 Folders and file organisation
- 2.3.7 Spreadsheets: rows, columns, simple formulas
- 2.3.8 Collecting and displaying data: charts in spreadsheet

2.4 Networks and Internet

- 2.4.1 What is the internet? A global network of computers
- 2.4.2 World Wide Web vs internet
- 2.4.3 Websites and web browsers
- 2.4.4 Search engines: how to search effectively
- 2.4.5 Evaluating websites: is it reliable?
- 2.4.6 Email: sending and receiving
- 2.4.7 Online safety: passwords, privacy, phishing
- 2.4.8 Copyright: using others' work fairly

** Scratch (scratch.mit.edu) is completely free. Code.org also has excellent free courses for this age.*

3.1 Python Basics

- 3.1.1 Why Python? -- readable, powerful, widely used
- 3.1.2 Installing Python and IDLE / VS Code
- 3.1.3 print() function: Hello, World!
- 3.1.4 Variables: storing data
- 3.1.5 Data types: int, float, str, bool
- 3.1.6 Input from user: input() function
- 3.1.7 Type conversion: int(), float(), str()
- 3.1.8 Arithmetic operators: +, -, *, /, //, %, **
- 3.1.9 String operations: concatenation, slicing, len()
- 3.1.10 String methods: .upper(), .lower(), .strip(), .replace(), .split()
- 3.1.11 f-strings: formatted output
- 3.1.12 Comments: # single line, triple quotes multi-line

3.2 Control Flow

- 3.2.1 Boolean expressions: ==, !=, >, <, >=, <=
- 3.2.2 Logical operators: and, or, not
- 3.2.3 if statement
- 3.2.4 if-else statement
- 3.2.5 if-elif-else chain
- 3.2.6 Nested if statements
- 3.2.7 while loop: condition-controlled repetition
- 3.2.8 for loop: count-controlled repetition
- 3.2.9 range() function
- 3.2.10 break and continue
- 3.2.11 Nested loops
- 3.2.12 Loop patterns: accumulator, counter, flag

3.3 Data Structures -- Basics

- 3.3.1 Lists: creating, indexing, slicing
- 3.3.2 List methods: append(), remove(), pop(), sort(), reverse(), len()
- 3.3.3 Iterating over a list with for loop
- 3.3.4 2D lists (lists of lists): basic matrices
- 3.3.5 Tuples: immutable sequences
- 3.3.6 Dictionaries: key-value pairs
- 3.3.7 Dictionary methods: .keys(), .values(), .items(), .get()
- 3.3.8 Sets: unique elements, union, intersection
- 3.3.9 When to use which data structure

3.4 Functions

- 3.4.1 Defining functions: def keyword
- 3.4.2 Parameters and arguments
- 3.4.3 Return values
- 3.4.4 Default parameter values
- 3.4.5 Local vs global variables: scope
- 3.4.6 Docstrings: documenting functions
- 3.4.7 Functions calling other functions
- 3.4.8 Recursion: a function calling itself
- 3.4.9 Recursive examples: factorial, Fibonacci
- 3.4.10 Base case and recursive case

3.5 Files and Exceptions

- 3.5.1 Reading a text file: `open()`, `read()`, `readlines()`
- 3.5.2 Writing to a file: `write()`, append mode
- 3.5.3 With statement: context manager
- 3.5.4 CSV files: reading and writing
- 3.5.5 Exception handling: `try`, `except`, `finally`
- 3.5.6 Common exceptions: `ValueError`, `TypeError`, `FileNotFoundError`
- 3.5.7 Raising exceptions

3.6 Introduction to Web

- 3.6.1 HTML: structure of a webpage
- 3.6.2 HTML tags: `h1-h6`, `p`, `a`, `img`, `ul`, `ol`, `li`, `table`, `div`, `span`
- 3.6.3 CSS: styling web pages
- 3.6.4 CSS selectors, properties, values
- 3.6.5 Box model: margin, border, padding, content
- 3.6.6 Colours, fonts, backgrounds in CSS
- 3.6.7 Simple webpage project: personal profile page

3.7 Number Systems and Data Representation

- 3.7.1 Binary number system: base 2
- 3.7.2 Converting decimal to binary and back
- 3.7.3 Hexadecimal: base 16
- 3.7.4 Converting between hex and binary
- 3.7.5 Representing text: ASCII, Unicode
- 3.7.6 Representing images: pixels, RGB, resolution
- 3.7.7 Representing sound: sampling rate, bit depth
- 3.7.8 File sizes: bits, bytes, KB, MB, GB, TB
- 3.7.9 Compression: lossless vs lossy

** [Python.org](https://python.org) official tutorial is free. CS50P (Harvard's free Python course on edX) is excellent for this level.*

4.1 Object-Oriented Programming (OOP)

- 4.1.1 Classes and objects: blueprint and instance
- 4.1.2 Attributes: instance variables
- 4.1.3 Methods: functions inside a class
- 4.1.4 `__init__` constructor method
- 4.1.5 `self` parameter
- 4.1.6 Encapsulation: public, protected, private
- 4.1.7 Inheritance: child class inherits parent class
- 4.1.8 Method overriding
- 4.1.9 `super()` function
- 4.1.10 Polymorphism: same method, different behaviour
- 4.1.11 Abstract classes and interfaces
- 4.1.12 Composition vs inheritance
- 4.1.13 Magic/dunder methods: `__str__`, `__len__`, `__eq__`
- 4.1.14 Class variables vs instance variables
- 4.1.15 Static and class methods
- 4.1.16 UML class diagrams

4.2 Data Structures -- Intermediate

- 4.2.1 Array vs list: fixed-size vs dynamic
- 4.2.2 Stack: LIFO, push, pop, peek
- 4.2.3 Queue: FIFO, enqueue, dequeue
- 4.2.4 Linked list: nodes, head, traversal
- 4.2.5 Singly linked list: insert, delete, search
- 4.2.6 Doubly linked list
- 4.2.7 Binary tree: root, nodes, leaves
- 4.2.8 Binary Search Tree (BST): insert, search, delete
- 4.2.9 Tree traversals: in-order, pre-order, post-order
- 4.2.10 Hash table: hashing, collision handling
- 4.2.11 Graph: vertices, edges, directed vs undirected
- 4.2.12 Graph representations: adjacency matrix, adjacency list

4.3 Algorithms -- Core

- 4.3.1 Algorithm analysis: Big O notation
- 4.3.2 $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$
- 4.3.3 Best, worst, average case
- 4.3.4 Space complexity
- 4.3.5 Searching: linear search $O(n)$
- 4.3.6 Binary search $O(\log n)$ -- sorted array required
- 4.3.7 Sorting: bubble sort -- $O(n^2)$
- 4.3.8 Sorting: insertion sort -- $O(n^2)$
- 4.3.9 Sorting: selection sort -- $O(n^2)$
- 4.3.10 Sorting: merge sort -- $O(n \log n)$, divide and conquer
- 4.3.11 Sorting: quicksort -- $O(n \log n)$ average
- 4.3.12 Sorting: heap sort
- 4.3.13 Graph traversal: BFS (Breadth-First Search)
- 4.3.14 Graph traversal: DFS (Depth-First Search)
- 4.3.15 Recursion vs iteration trade-offs

4.4 Computer Systems

- 4.4.1 Von Neumann architecture: CPU, memory, I/O, bus

- 4.4.2 CPU components: ALU, Control Unit, registers
- 4.4.3 Fetch-decode-execute cycle
- 4.4.4 Primary memory: RAM and ROM
- 4.4.5 Secondary storage: HDD, SSD, optical, flash
- 4.4.6 Cache memory: L1, L2, L3
- 4.4.7 Clock speed and processor performance
- 4.4.8 Multi-core processors
- 4.4.9 Operating systems: functions, types
- 4.4.10 Memory management: allocation, virtual memory, paging
- 4.4.11 Process management: scheduling, multitasking
- 4.4.12 File systems: FAT, NTFS, ext4
- 4.4.13 Device drivers
- 4.4.14 Interrupts
- 4.4.15 BIOS and boot process

4.5 Networks -- IGCSE

- 4.5.1 LAN, WAN, MAN, PAN
- 4.5.2 Network topologies: star, bus, ring, mesh
- 4.5.3 Wired vs wireless: Ethernet, Wi-Fi, Bluetooth
- 4.5.4 Network hardware: router, switch, hub, access point, NIC
- 4.5.5 IP addresses: IPv4, IPv6
- 4.5.6 MAC addresses
- 4.5.7 DNS: domain names to IP addresses
- 4.5.8 DHCP: automatic IP assignment
- 4.5.9 Protocols: TCP/IP, HTTP, HTTPS, FTP, SMTP, POP3, IMAP
- 4.5.10 TCP/IP layers: application, transport, internet, link
- 4.5.11 Packet switching
- 4.5.12 Encryption: symmetric, asymmetric, SSL/TLS
- 4.5.13 Firewalls and network security
- 4.5.14 VPN
- 4.5.15 Cloud computing: IaaS, PaaS, SaaS

4.6 Databases

- 4.6.1 Why databases? Problems with flat files
- 4.6.2 Relational database model: tables, rows, columns
- 4.6.3 Primary key and foreign key
- 4.6.4 Relationships: one-to-one, one-to-many, many-to-many
- 4.6.5 Entity-Relationship (ER) diagrams
- 4.6.6 Normalisation: 1NF, 2NF, 3NF
- 4.6.7 SQL: SELECT, FROM, WHERE, ORDER BY, GROUP BY
- 4.6.8 SQL: INSERT, UPDATE, DELETE
- 4.6.9 SQL: JOIN (INNER, LEFT, RIGHT, FULL)
- 4.6.10 SQL: aggregate functions: COUNT, SUM, AVG, MAX, MIN
- 4.6.11 SQL: CREATE TABLE, DROP TABLE, ALTER TABLE
- 4.6.12 Transactions: ACID properties
- 4.6.13 Indexes: speeding up queries
- 4.6.14 Views
- 4.6.15 Database security: user permissions

4.7 Cybersecurity

- 4.7.1 Threats: malware, virus, worm, trojan, ransomware, spyware
- 4.7.2 Social engineering: phishing, vishing, smishing
- 4.7.3 Network attacks: DoS, DDoS, man-in-the-middle, SQL injection

- 4.7.4 Password security: hashing, salting, strength
- 4.7.5 Two-factor authentication (2FA)
- 4.7.6 Cryptography: Caesar cipher, symmetric, asymmetric
- 4.7.7 Public key infrastructure (PKI)
- 4.7.8 Digital signatures and certificates
- 4.7.9 HTTPS and SSL/TLS
- 4.7.10 Penetration testing concept
- 4.7.11 Ethical and legal issues in computing
- 4.7.12 Intellectual property: copyright, patents, open source
- 4.7.13 Data protection laws: GDPR concept

** Cambridge IGCSE CS (0478), CS50 (Harvard free course on edX), and Python projects -- best combo for this level.*

5.1 Advanced Programming Concepts

- 5.1.1 Higher-order functions: map(), filter(), reduce()
- 5.1.2 Lambda functions
- 5.1.3 List comprehensions and generator expressions
- 5.1.4 Decorators
- 5.1.5 Generators and iterators: yield
- 5.1.6 Context managers: with statement deep dive
- 5.1.7 Regular expressions: pattern matching
- 5.1.8 Modules and packages: import, pip
- 5.1.9 Virtual environments
- 5.1.10 Unit testing: unittest, pytest
- 5.1.11 Debugging techniques and tools
- 5.1.12 Version control: Git -- init, add, commit, push, pull, branch, merge
- 5.1.13 GitHub: repositories, pull requests, issues
- 5.1.14 Documentation: comments, docstrings, README
- 5.1.15 Code style: PEP 8

5.2 Advanced Data Structures

- 5.2.1 Heap: max-heap, min-heap, heapify
- 5.2.2 Priority queue
- 5.2.3 AVL tree: self-balancing BST
- 5.2.4 Red-black tree
- 5.2.5 B-tree and B+ tree (used in databases)
- 5.2.6 Trie: prefix tree for strings
- 5.2.7 Graph advanced: weighted graphs, DAGs
- 5.2.8 Dijkstra's shortest path algorithm
- 5.2.9 Bellman-Ford algorithm
- 5.2.10 Floyd-Warshall: all-pairs shortest paths
- 5.2.11 Minimum spanning tree: Kruskal's, Prim's algorithms
- 5.2.12 Topological sort
- 5.2.13 Union-Find (Disjoint Set Union)
- 5.2.14 Segment tree
- 5.2.15 Fenwick tree (Binary Indexed Tree)

5.3 Algorithm Design Paradigms

- 5.3.1 Divide and conquer: merge sort, quick sort, binary search
- 5.3.2 Dynamic programming: concept, memoisation, tabulation
- 5.3.3 DP: Fibonacci, coin change, longest common subsequence
- 5.3.4 DP: 0/1 knapsack, matrix chain multiplication
- 5.3.5 Greedy algorithms: concept and proof of correctness
- 5.3.6 Greedy: activity selection, Huffman coding, fractional knapsack
- 5.3.7 Backtracking: N-queens, Sudoku, permutations
- 5.3.8 Branch and bound
- 5.3.9 Amortised analysis
- 5.3.10 Randomised algorithms: quickselect, randomised quicksort

5.4 Computer Architecture -- Advanced

- 5.4.1 Instruction set architecture (ISA): RISC vs CISC
- 5.4.2 Assembly language basics: registers, load, store, jump
- 5.4.3 Pipeline architecture: fetch, decode, execute, memory, writeback

- 5.4.4 Pipeline hazards: data, control, structural
- 5.4.5 Out-of-order execution
- 5.4.6 Superscalar processors
- 5.4.7 Memory hierarchy: registers, cache, RAM, disk
- 5.4.8 Cache design: direct-mapped, set-associative, fully associative
- 5.4.9 Cache replacement policies: LRU, FIFO
- 5.4.10 Virtual memory: page tables, TLB
- 5.4.11 DMA (Direct Memory Access)
- 5.4.12 GPU architecture basics
- 5.4.13 Parallel computing: SIMD, MIMD
- 5.4.14 Boolean algebra: logic gates, circuits
- 5.4.15 Combinational circuits: adder, multiplexer, decoder
- 5.4.16 Sequential circuits: flip-flops, registers, counters

5.5 Theory of Computation -- Introduction

- 5.5.1 Finite automata (DFA): states, transitions, acceptance
- 5.5.2 Non-deterministic finite automata (NFA)
- 5.5.3 Regular expressions and regular languages
- 5.5.4 Pumping lemma for regular languages
- 5.5.5 Context-free grammars (CFG)
- 5.5.6 Push-down automata (PDA)
- 5.5.7 Chomsky Normal Form
- 5.5.8 Parsing: top-down and bottom-up
- 5.5.9 Turing machines: concept and operation
- 5.5.10 Church-Turing thesis
- 5.5.11 Decidability: decidable vs undecidable problems
- 5.5.12 Halting problem: proof of undecidability
- 5.5.13 Reductions between problems
- 5.5.14 Complexity classes: P, NP
- 5.5.15 NP-completeness: SAT, 3-SAT, travelling salesman
- 5.5.16 P vs NP question

5.6 Software Engineering

- 5.6.1 Software development lifecycle (SDLC)
- 5.6.2 Waterfall model
- 5.6.3 Agile methodologies: Scrum, Kanban
- 5.6.4 Requirements engineering: functional and non-functional
- 5.6.5 System design: top-down and bottom-up
- 5.6.6 UML diagrams: class, sequence, use case, activity
- 5.6.7 Design patterns: Singleton, Factory, Observer, MVC
- 5.6.8 Testing: unit, integration, system, acceptance
- 5.6.9 Test-driven development (TDD)
- 5.6.10 Code review and pair programming
- 5.6.11 Software maintenance and refactoring
- 5.6.12 DevOps and CI/CD pipelines

5.7 Introduction to AI and Machine Learning

- 5.7.1 What is AI? History and definition
- 5.7.2 Search algorithms: BFS, DFS, A* for AI
- 5.7.3 Game AI: minimax algorithm
- 5.7.4 Alpha-beta pruning
- 5.7.5 Machine learning: supervised, unsupervised, reinforcement
- 5.7.6 Linear regression

- 5.7.7 Logistic regression and classification
- 5.7.8 Decision trees and random forests
- 5.7.9 k-Nearest Neighbours (kNN)
- 5.7.10 Neural network: perceptron, layers, activation functions
- 5.7.11 Training: forward pass, loss function, backpropagation
- 5.7.12 Overfitting and underfitting: bias-variance trade-off
- 5.7.13 Validation and test sets, cross-validation
- 5.7.14 Practical: scikit-learn for simple ML projects

** CS50 (Harvard), A-Level CS Cambridge (9618), USACO competitive programming -- all excellent at this level.*

6.1 Discrete Mathematics for CS

- 6.1.1 Propositional logic: connectives, truth tables, tautologies
- 6.1.2 Predicate logic: quantifiers, proofs
- 6.1.3 Set theory: operations, power set, Cartesian product
- 6.1.4 Functions: injective, surjective, bijective, composition
- 6.1.5 Relations: equivalence, partial order
- 6.1.6 Proof techniques: direct, contrapositive, contradiction, induction
- 6.1.7 Strong induction and structural induction
- 6.1.8 Combinatorics: permutations, combinations, pigeonhole
- 6.1.9 Generating functions
- 6.1.10 Recurrence relations: solving with iteration and characteristic equation
- 6.1.11 Master theorem for divide-and-conquer recurrences
- 6.1.12 Graph theory: paths, cycles, trees, planarity, colouring
- 6.1.13 Probability: sample spaces, conditional probability, Bayes' theorem
- 6.1.14 Random variables, expectation, variance
- 6.1.15 Distributions: binomial, geometric, Poisson, normal
- 6.1.16 Markov chains
- 6.1.17 Number theory for CS: modular arithmetic, GCD, RSA

6.2 Advanced Algorithms

- 6.2.1 Advanced sorting: radix sort, counting sort, bucket sort
- 6.2.2 String algorithms: KMP, Rabin-Karp, Boyer-Moore
- 6.2.3 Suffix arrays and suffix trees
- 6.2.4 Advanced graph algorithms: strongly connected components (Tarjan, Kosaraju)
- 6.2.5 Network flow: Ford-Fulkerson, max-flow min-cut theorem
- 6.2.6 Bipartite matching: Hungarian algorithm
- 6.2.7 Advanced DP: bitmask DP, tree DP, interval DP
- 6.2.8 Computational geometry: convex hull, line intersection
- 6.2.9 External memory algorithms
- 6.2.10 Cache-oblivious algorithms
- 6.2.11 Streaming algorithms: approximation
- 6.2.12 Approximation algorithms: vertex cover, TSP
- 6.2.13 Parameterised complexity
- 6.2.14 Online algorithms: competitive ratio

6.3 Operating Systems

- 6.3.1 OS structure: monolithic, microkernel, layered, modular
- 6.3.2 System calls: interface between user and kernel
- 6.3.3 Processes: PCB, process states, creation, termination
- 6.3.4 Threads: user-level vs kernel-level
- 6.3.5 CPU scheduling: FCFS, SJF, round robin, priority, multilevel queue
- 6.3.6 Process synchronisation: race conditions, critical section
- 6.3.7 Semaphores and mutexes
- 6.3.8 Classic problems: producer-consumer, readers-writers, dining philosophers
- 6.3.9 Deadlock: conditions, prevention, avoidance (Banker's algorithm), detection
- 6.3.10 Memory management: contiguous allocation, fragmentation
- 6.3.11 Paging: page tables, TLB, page faults
- 6.3.12 Segmentation
- 6.3.13 Virtual memory: demand paging, page replacement (FIFO, LRU, optimal)
- 6.3.14 Thrashing
- 6.3.15 File system implementation: FAT, inode, directories

- 6.3.16 I/O management: polling, interrupts, DMA
- 6.3.17 Mass storage: disk scheduling (FCFS, SSTF, SCAN, C-SCAN)
- 6.3.18 RAID levels
- 6.3.19 Linux internals: kernel, processes, files

6.4 Computer Networks

- 6.4.1 OSI model: 7 layers and their functions
- 6.4.2 TCP/IP model: 4 layers
- 6.4.3 Physical layer: encoding, transmission media
- 6.4.4 Data link layer: framing, error detection (CRC), MAC
- 6.4.5 Multiple access protocols: CSMA/CD, CSMA/CA
- 6.4.6 Ethernet: frames, switches
- 6.4.7 Network layer: IP, routing, subnetting, CIDR
- 6.4.8 Routing protocols: RIP, OSPF, BGP
- 6.4.9 NAT and private addressing
- 6.4.10 Transport layer: TCP (reliable, connection-oriented) vs UDP
- 6.4.11 TCP: 3-way handshake, flow control, congestion control
- 6.4.12 Application layer: HTTP/HTTPS, DNS, SMTP, FTP, SSH
- 6.4.13 Socket programming: client-server model
- 6.4.14 Wireless networking: 802.11 standards
- 6.4.15 Software-defined networking (SDN)
- 6.4.16 Content delivery networks (CDNs)

6.5 Advanced Databases

- 6.5.1 Advanced SQL: subqueries, window functions, CTEs
- 6.5.2 Query optimisation: query plan, indexes, cost model
- 6.5.3 Transactions: ACID deep dive
- 6.5.4 Concurrency control: locking (2PL), MVCC
- 6.5.5 Isolation levels: read uncommitted, read committed, repeatable read, serialisable
- 6.5.6 Recovery: log-based recovery, ARIES protocol
- 6.5.7 Database design: ER to relational mapping
- 6.5.8 NoSQL databases: document (MongoDB), key-value (Redis), column (Cassandra), graph (Neo4j)
- 6.5.9 CAP theorem: consistency, availability, partition tolerance
- 6.5.10 BASE vs ACID
- 6.5.11 Distributed databases: sharding, replication
- 6.5.12 Data warehousing: OLAP vs OLTP, star schema
- 6.5.13 Big data: Hadoop, Spark -- overview

6.6 Programming Languages

- 6.6.1 Paradigms: imperative, declarative, functional, logic, OOP
- 6.6.2 Type systems: static vs dynamic, strong vs weak
- 6.6.3 Compiled vs interpreted languages
- 6.6.4 Memory models: stack, heap, garbage collection
- 6.6.5 C programming: pointers, memory allocation, arrays, structs
- 6.6.6 C: manual memory management: malloc, free
- 6.6.7 C++: classes, templates, STL, RAII
- 6.6.8 Java: JVM, generics, concurrency
- 6.6.9 JavaScript: event loop, closures, prototypes, async/await
- 6.6.10 Functional programming: Haskell or OCaml -- pure functions, immutability
- 6.6.11 Haskell: type classes, monads, lazy evaluation
- 6.6.12 Rust: ownership, borrowing, lifetimes, memory safety without GC
- 6.6.13 Scripting: Bash/shell scripting
- 6.6.14 Concurrency models: threads, async, actors, CSP

* MIT 6.006 (Algorithms), 6.033 (Systems), 6.004 (Architecture) -- all free on OCW. CLRS textbook is the algorithms bible.

7.1 Compilers and Programming Language Theory

- 7.1.1 Compiler phases: lexing, parsing, semantic analysis, IR, optimisation, code gen
- 7.1.2 Lexical analysis: regular expressions, finite automata, tokenisation
- 7.1.3 Syntax analysis: CFG, LL(1) parsing, LR parsing
- 7.1.4 Abstract Syntax Tree (AST)
- 7.1.5 Semantic analysis: type checking, symbol table
- 7.1.6 Intermediate representations: three-address code, SSA form
- 7.1.7 Code optimisation: constant folding, dead code elimination, loop optimisation
- 7.1.8 Register allocation: graph colouring
- 7.1.9 Code generation: instruction selection
- 7.1.10 Runtime systems: stack frames, calling conventions
- 7.1.11 Garbage collection algorithms: mark-and-sweep, reference counting, generational
- 7.1.12 Type theory: type inference, Hindley-Milner
- 7.1.13 Lambda calculus: alpha, beta, eta reduction
- 7.1.14 Curry-Howard correspondence: proofs as programs
- 7.1.15 Dependent types: Coq, Agda -- brief
- 7.1.16 JIT compilation
- 7.1.17 LLVM infrastructure

7.2 Distributed Systems

- 7.2.1 Distributed systems: challenges (partial failures, async networks)
- 7.2.2 Remote procedure calls (RPC)
- 7.2.3 Time and clocks: Lamport timestamps, vector clocks
- 7.2.4 Consistency models: sequential, linearisability, eventual consistency
- 7.2.5 Replication: primary-backup, multi-leader, leaderless
- 7.2.6 Consensus: Paxos algorithm
- 7.2.7 Raft consensus algorithm
- 7.2.8 Fault tolerance: Byzantine generals problem
- 7.2.9 Byzantine fault tolerance (BFT)
- 7.2.10 Distributed transactions: 2-phase commit
- 7.2.11 Google Spanner and F1
- 7.2.12 MapReduce and Hadoop
- 7.2.13 Apache Spark
- 7.2.14 Stream processing: Kafka, Flink
- 7.2.15 Microservices architecture
- 7.2.16 Containerisation: Docker, Kubernetes
- 7.2.17 Load balancing and service discovery
- 7.2.18 Distributed file systems: GFS, HDFS

7.3 Machine Learning -- Full Course

- 7.3.1 Linear algebra for ML: vectors, matrices, eigendecomposition, SVD
- 7.3.2 Probability and statistics for ML: MLE, MAP, Bayesian inference
- 7.3.3 Linear regression: closed form, gradient descent
- 7.3.4 Logistic regression: sigmoid, cross-entropy loss
- 7.3.5 Regularisation: L1 (Lasso), L2 (Ridge), Elastic Net
- 7.3.6 Support Vector Machines (SVM): hard and soft margin, kernel trick
- 7.3.7 Decision trees: information gain, Gini impurity
- 7.3.8 Ensemble methods: bagging, boosting, random forests, XGBoost
- 7.3.9 Neural networks: architecture, activation functions (ReLU, sigmoid, tanh, softmax)
- 7.3.10 Backpropagation: chain rule, gradient computation

- 7.3.11 Optimisation: SGD, Momentum, Adam, learning rate schedules
- 7.3.12 Regularisation in NN: dropout, batch normalisation, weight decay
- 7.3.13 Convolutional Neural Networks (CNNs): convolution, pooling, architectures
- 7.3.14 CNN architectures: LeNet, AlexNet, VGG, ResNet, Inception
- 7.3.15 Recurrent Neural Networks (RNNs): BPTT, vanishing gradient
- 7.3.16 LSTM and GRU
- 7.3.17 Attention mechanism
- 7.3.18 Transformer architecture: self-attention, multi-head attention, positional encoding
- 7.3.19 Transfer learning and fine-tuning
- 7.3.20 Unsupervised learning: k-means, hierarchical clustering
- 7.3.21 Dimensionality reduction: PCA, t-SNE, UMAP
- 7.3.22 Autoencoders: vanilla, VAE (Variational Autoencoder)
- 7.3.23 Generative Adversarial Networks (GANs)
- 7.3.24 Reinforcement Learning: MDP, Bellman equations
- 7.3.25 RL algorithms: Q-learning, SARSA, DQN, policy gradients
- 7.3.26 Evaluation metrics: accuracy, precision, recall, F1, ROC-AUC, BLEU
- 7.3.27 Feature engineering and selection
- 7.3.28 Handling imbalanced data: SMOTE, class weights
- 7.3.29 ML pipelines and MLOps

7.4 Computer Graphics and Vision

- 7.4.1 2D graphics: coordinate systems, transformations
- 7.4.2 3D graphics: homogeneous coordinates, 4x4 matrices
- 7.4.3 Rendering pipeline: vertex processing, rasterisation, fragment processing
- 7.4.4 OpenGL/WebGL basics
- 7.4.5 Lighting models: Phong, Blinn-Phong
- 7.4.6 Texture mapping
- 7.4.7 Ray tracing: primary rays, shadows, reflections
- 7.4.8 Path tracing and global illumination
- 7.4.9 Computer vision: image representation, convolution
- 7.4.10 Feature detection: SIFT, SURF, ORB
- 7.4.11 Object detection: YOLO, Faster R-CNN
- 7.4.12 Image segmentation: semantic, instance
- 7.4.13 Face recognition
- 7.4.14 Optical flow and video understanding
- 7.4.15 3D vision: stereo, depth estimation, point clouds
- 7.4.16 Diffusion models for image generation

7.5 Advanced Security

- 7.5.1 Cryptography: symmetric (AES), asymmetric (RSA, ECC)
- 7.5.2 Hash functions: SHA-256, SHA-3, collision resistance
- 7.5.3 Digital signatures: ECDSA
- 7.5.4 Key exchange: Diffie-Hellman, ECDH
- 7.5.5 Zero-knowledge proofs
- 7.5.6 Post-quantum cryptography
- 7.5.7 Side-channel attacks: timing, power analysis
- 7.5.8 Buffer overflow and stack smashing
- 7.5.9 Return-oriented programming (ROP)
- 7.5.10 Web security: OWASP Top 10, XSS, CSRF, SQL injection
- 7.5.11 Reverse engineering and malware analysis
- 7.5.12 Network forensics
- 7.5.13 Formal verification of security protocols
- 7.5.14 Privacy-preserving computation: MPC, homomorphic encryption

7.5.15 Blockchain: Merkle trees, consensus, smart contracts

* *CLRS (Algorithms), Tanenbaum (OS + Networks), Goodfellow (Deep Learning) -- these are the standard textbooks.*

8.1 Advanced Theory of Computation

- 8.1.1 Complexity theory: space complexity classes PSPACE, EXPTIME
- 8.1.2 Polynomial hierarchy: Sigma, Pi, Delta classes
- 8.1.3 Circuit complexity: NC, AC classes
- 8.1.4 Randomised complexity: BPP, RP, ZPP
- 8.1.5 Interactive proofs: IP = PSPACE (Shamir)
- 8.1.6 Probabilistically checkable proofs (PCP theorem)
- 8.1.7 Hardness of approximation: inapproximability
- 8.1.8 Communication complexity
- 8.1.9 Decision tree complexity
- 8.1.10 Descriptive complexity: Fagin's theorem
- 8.1.11 Kolmogorov complexity and Solomonoff induction
- 8.1.12 Type theory: System F, dependent types, HoTT
- 8.1.13 Category theory for CS: functors, monads
- 8.1.14 Domain theory: denotational semantics
- 8.1.15 Model checking: temporal logics, CTL, LTL
- 8.1.16 Separation logic for program verification
- 8.1.17 Automated theorem proving: SAT solvers, SMT solvers

8.2 Advanced AI and Deep Learning

- 8.2.1 Large language models (LLMs): GPT architecture deep dive
- 8.2.2 BERT, T5, and encoder-decoder models
- 8.2.3 Instruction tuning and RLHF
- 8.2.4 In-context learning and chain-of-thought prompting
- 8.2.5 Scaling laws: Chinchilla, emergent abilities
- 8.2.6 Multimodal models: CLIP, DALL-E, Flamingo
- 8.2.7 Diffusion models: DDPM, stable diffusion -- theory
- 8.2.8 Graph neural networks (GNNs): message passing, spectral
- 8.2.9 Geometric deep learning: equivariant networks
- 8.2.10 Neural architecture search (NAS)
- 8.2.11 Advanced RL: PPO, SAC, model-based RL, multi-agent RL
- 8.2.12 Causal inference and causal ML
- 8.2.13 Probabilistic graphical models: Bayesian networks, MRFs
- 8.2.14 Variational inference: ELBO, mean field
- 8.2.15 Monte Carlo methods: MCMC, HMC
- 8.2.16 Gaussian processes
- 8.2.17 Interpretability and explainability: LIME, SHAP, attention analysis
- 8.2.18 Robustness: adversarial examples, certified defences
- 8.2.19 Fairness in ML: metrics, mitigation
- 8.2.20 AI safety and alignment: reward hacking, specification gaming

8.3 Advanced Systems

- 8.3.1 Advanced OS: microkernel design, Sel4 verified OS
- 8.3.2 Real-time systems: scheduling, WCET analysis
- 8.3.3 Embedded systems programming
- 8.3.4 Systems performance engineering: profiling, tracing, benchmarking
- 8.3.5 Memory-safe systems programming: Rust advanced
- 8.3.6 LLVM internals and pass writing
- 8.3.7 Hardware/software co-design
- 8.3.8 FPGA programming
- 8.3.9 High-performance computing: MPI, OpenMP, CUDA

- 8.3.10 GPU programming: CUDA architecture, optimisation
- 8.3.11 Database internals: buffer pool, B+ trees, WAL
- 8.3.12 NewSQL: CockroachDB, TiDB
- 8.3.13 Cloud-native architectures: serverless, functions-as-a-service
- 8.3.14 Observability: metrics, logs, traces -- Prometheus, Grafana, Jaeger

8.4 Advanced Networks and Security

- 8.4.1 Internet architecture: BGP internals, peering, IXPs
- 8.4.2 5G networks: architecture and technologies
- 8.4.3 Network virtualisation: NFV, SDN advanced
- 8.4.4 P4 programmable data planes
- 8.4.5 Network measurement at scale
- 8.4.6 Formal methods in security: protocol verification (ProVerif, Tamarin)
- 8.4.7 Binary analysis: symbolic execution, fuzzing
- 8.4.8 Vulnerability research and CVE process
- 8.4.9 Trusted execution environments: Intel SGX, ARM TrustZone
- 8.4.10 Differential privacy: formal definition, mechanisms, applications
- 8.4.11 Federated learning and privacy

8.5 Quantum Computing

- 8.5.1 Quantum bits (qubits): superposition, measurement
- 8.5.2 Quantum gates: X, Y, Z, H, CNOT, Toffoli
- 8.5.3 Quantum circuits and universality
- 8.5.4 Entanglement and Bell states
- 8.5.5 Quantum teleportation
- 8.5.6 Quantum algorithms: Deutsch-Jozsa, Bernstein-Vazirani
- 8.5.7 Grover's search algorithm: $O(\sqrt{N})$ speedup
- 8.5.8 Shor's factoring algorithm: exponential speedup
- 8.5.9 Quantum error correction: stabiliser codes, surface codes
- 8.5.10 Quantum complexity: BQP, QMA
- 8.5.11 Quantum machine learning
- 8.5.12 Physical implementations: superconducting, trapped ion, photonic
- 8.5.13 Noisy Intermediate-Scale Quantum (NISQ) devices
- 8.5.14 Variational quantum eigensolver (VQE), QAOA
- 8.5.15 Quantum cryptography: BB84, quantum key distribution

* *CMU 15-751, MIT 6.840 (Theory), Stanford CS229 (ML), Berkeley CS285 (RL) -- world-class free courses.*

CHAPTER 9 -- PhD / RESEARCH FRONTIER (Level 9 | MIT . Stanford . CMU . DeepMind)

Ye wo topics hain jahan original CS research hoti hai. MIT CSAIL, Stanford AI Lab, CMU SCS, Berkeley EECS, Oxford CS, Cambridge Engineering, DeepMind, Google Brain, OpenAI, Meta FAIR -- yahan researchers naya knowledge create karte hain.

9.1 Theoretical Computer Science Frontier

- 9.1.1 P vs NP -- current approaches and barriers
- 9.1.2 Geometric complexity theory (Mulmuley)
- 9.1.3 Algebraic complexity theory
- 9.1.4 Fine-grained complexity: SETH, 3SUM conjecture
- 9.1.5 Derandomisation: BPP vs P debate
- 9.1.6 Pseudorandomness: PRGs, expander graphs, extractors
- 9.1.7 Coding theory: list decoding, locally decodable codes
- 9.1.8 Cryptographic hardness assumptions: LWE, SIS
- 9.1.9 Obfuscation and functional encryption
- 9.1.10 Proof complexity
- 9.1.11 Sublinear algorithms
- 9.1.12 Property testing

9.2 AI and ML Research Frontier

- 9.2.1 Foundation models: scaling, emergent abilities, capabilities vs alignment
- 9.2.2 AI alignment: RLHF, Constitutional AI, debate, amplification
- 9.2.3 Interpretability: mechanistic interpretability (Anthropic), superposition
- 9.2.4 AI agents: tool use, planning, memory, multi-agent systems
- 9.2.5 World models and model-based RL
- 9.2.6 Sample efficiency in RL
- 9.2.7 Neuroscience-inspired AI: predictive coding, sparse coding
- 9.2.8 AI for science: AlphaFold, protein design, drug discovery
- 9.2.9 Reasoning in LLMs: formal verification of neural networks
- 9.2.10 Compositional generalisation
- 9.2.11 Continual learning / lifelong learning
- 9.2.12 Zero-shot and few-shot learning
- 9.2.13 Self-supervised learning: contrastive, masked autoencoding
- 9.2.14 Multimodal learning frontier
- 9.2.15 AI for code: Codex, AlphaCode, program synthesis

9.3 Systems Research Frontier

- 9.3.1 OS research: Barrelfish, Arrakis, IX
- 9.3.2 New hardware: CXL interconnect, NVM, near-data processing
- 9.3.3 Datacenter-scale computing: warehouse-scale computers
- 9.3.4 The Datacenter as a Computer (Barroso, Holzle)
- 9.3.5 Network operating systems
- 9.3.6 Persistent memory programming
- 9.3.7 Disaggregated datacenter architectures
- 9.3.8 ML systems: PyTorch internals, TensorFlow XLA, custom accelerators (TPU)
- 9.3.9 Large-scale training systems: pipeline, tensor, data parallelism
- 9.3.10 Storage systems for ML
- 9.3.11 Verified systems: CompCert, CertiKOS, Sel4

9.4 Security and Privacy Research Frontier

- 9.4.1 LLM security: jailbreaks, prompt injection, data poisoning

- 9.4.2 Hardware security: Meltdown, Spectre, Rowhammer
- 9.4.3 Supply chain security
- 9.4.4 Fully homomorphic encryption (FHE) -- practical advances
- 9.4.5 Secure multi-party computation at scale
- 9.4.6 Differential privacy in practice: Apple, Google deployments
- 9.4.7 Privacy attacks on ML: membership inference, model inversion
- 9.4.8 Decentralised identity and verifiable credentials
- 9.4.9 Security of autonomous systems and CPS

9.5 Networks and Distributed Systems Frontier

- 9.5.1 Programmable networks research
- 9.5.2 QUIC protocol and HTTP/3
- 9.5.3 Named Data Networking (NDN)
- 9.5.4 Internet censorship and circumvention
- 9.5.5 Web3 and decentralised applications: limitations and research
- 9.5.6 Consistency in geo-distributed systems
- 9.5.7 CRDTs (Conflict-free Replicated Data Types)
- 9.5.8 Serverless and function composition
- 9.5.9 Edge computing and IoT at scale

9.6 Human-Computer Interaction (HCI) Frontier

- 9.6.1 Conversational AI interfaces
- 9.6.2 Augmented and mixed reality: rendering, interaction
- 9.6.3 Accessibility and universal design
- 9.6.4 Intelligent user interfaces
- 9.6.5 AI ethics and fairness in deployed systems
- 9.6.6 Measurement of human performance
- 9.6.7 Brain-computer interfaces

9.7 Quantum Computing Research Frontier

- 9.7.1 Fault-tolerant quantum computing: threshold theorem
- 9.7.2 Surface code implementation
- 9.7.3 Quantum advantage demonstrations
- 9.7.4 Quantum simulation of chemistry and materials
- 9.7.5 Quantum optimisation
- 9.7.6 Quantum communication networks
- 9.7.7 New quantum algorithms for graph problems

9.8 Bioinformatics and Computational Biology

- 9.8.1 Sequence alignment: Smith-Waterman, BLAST
- 9.8.2 Genome assembly algorithms
- 9.8.3 Protein structure prediction: AlphaFold2 -- architecture
- 9.8.4 Protein design: RFDiffusion, ProteinMPNN
- 9.8.5 Drug discovery with ML
- 9.8.6 Single-cell RNA sequencing analysis
- 9.8.7 Phylogenetics and evolutionary computation
- 9.8.8 Brain simulation and connectomics

9.9 Open Grand Challenges in CS

- 9.9.1 P vs NP: the greatest open problem in mathematics/CS
- 9.9.2 Can we build truly safe and aligned AI?
- 9.9.3 Can we build a fault-tolerant quantum computer?
- 9.9.4 Can we formally verify all software?
- 9.9.5 How do large language models actually work?

- 9.9.6 Can AI achieve human-level common sense reasoning?
- 9.9.7 How to build privacy-preserving systems at internet scale?
- 9.9.8 Can we make internet infrastructure truly resilient?
- 9.9.9 What is the minimum energy needed for computation? (Landauer)
- 9.9.10 Can we build an artificial general intelligence (AGI)?

COMPLETE ROADMAP -- AT A GLANCE

Chap.	Level	Age	Core Areas	Topics
1	L1	4-7	Digital literacy, computational thinking, online safety	~30
2	L2	7-10	Scratch/block coding, algorithms, networks, data	~60
3	L3	10-13	Python basics, control flow, data structures, HTML/CSS, binary	~75
4	L4	13-16	OOP, DSA, algorithms, computer systems, networks, databases, security	~130
5	L5	16-18	Advanced programming, advanced algorithms, architecture, theory, AI intro, SE	~125
6	L6	18-21	Discrete maths, advanced algorithms, OS, networks, databases, PLs	~130
7	L7	21-22	Compilers, distributed systems, full ML course, graphics, security	~125
8	L8	22-24	Advanced theory, LLMs, systems, quantum computing, security	~110
9	L9	24+	PhD frontier: AI safety, P vs NP, quantum, bioinformatics	~90
TOTAL				~875+ Topics

Yeh roadmap MIT EECS, Stanford CS, CMU SCS, Oxford CS, aur Cambridge Engineering ke syllabi par based hai. Scratch se Quantum Computing tak -- har topic pichle topic ki bunyaad par rakha gaya hai. Sabse important baat: CODE LIKHO. Padhna kaafi nahi -- roz kuch banao.